

STRUCTURE-AWARE FINANCIAL FORECASTING USING HANKEL MATRICES AND DUAL LSTM FUSION

A PROJECT REPORT

Submitted by

Dhwanit Sharma (DL.AI.U4AID24111)

Kanan Goel (DL.AI.U4AID24017)

Parkhi Mahajan (DL.AI.U4AID24126)

Punit Lohan (DL.AI.U4AID24127)

in partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY (B.Tech) IN ARTIFICIAL
INTELLIGENCE AND DATA SCIENCE (AIDS)**

Under the guidance of

Dr. Anirban Tarafdar

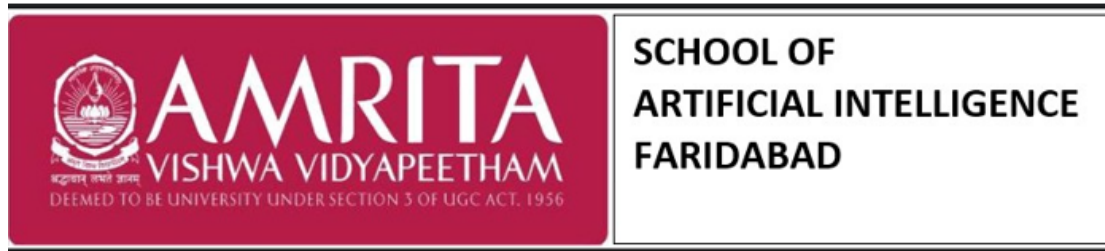
Submitted to



**AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE**

FARIDABAD – 121002

December 2025



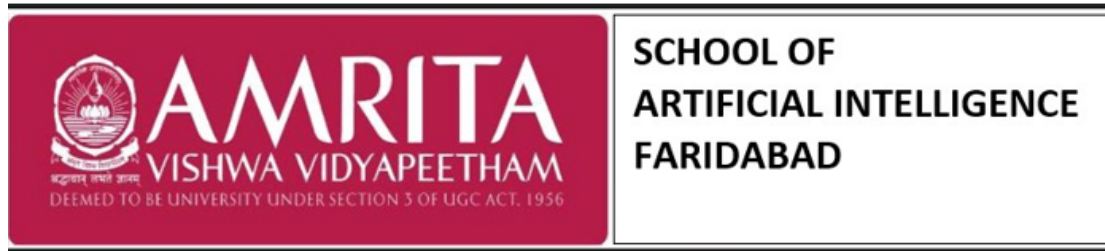
BONAFIDE CERTIFICATE

This is to certify that this project report entitled “**Structure-Aware Financial Forecasting using Hankel Matrices and Dual LSTM Fusion**” is the bonafide work of Dhwanit Sharma (DL.AI.U4AID24111), Kanan Goel (DL.AI.U4AID24017), Parkhi Mahajan (DL.AI.U4AID24126), Punit Lohan (DL.AI.U4AID24127), who carried out the project work under my supervision.

Dr. Anirban Tarafdar (Assistant Professor)

School of AI

Faridabad



DECLARATION BY THE CANDIDATE

I declare that the report entitled “**Structure-Aware Financial Forecasting using Hankel Matrices and Dual LSTM Fusion**” submitted by me for the degree of Bachelor of Technology is the record of the project work carried out by me under the guidance of **Dr. Anirban Tarafdar** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or title in this or any other University or other similar institution of higher learning.

Dhwanit Sharma (DL.AI.U4AID24111)

Kanan Goel (DL.AI.U4AID24017)

Parkhi Mahajan (DL.AI.U4AID24126)

Punit Lohan (DL.AI.U4AID24127)

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honorable Chancellor **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. I am indebted to extend my gratitude to our Principal, **Dr. Lakshmi Mohandas**, Amrita School of Computing and Engineering, for facilitating us with all the facilities and extended support to gain valuable education and learning experience.

I wish to express my sincere gratitude to my supervisor(s) **Dr. Anirban Tarafdar** for their personal involvement and constant encouragement during the entire course of this work.

I am grateful to the Project Supervisor, Review Panel Members, and the entire faculty of the Department of Artificial Intelligence and Data Science for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

Dhwanit Sharma (DL.AI.U4AID24111)

Kanan Goel (DL.AI.U4AID24017)

Parkhi Mahajan (DL.AI.U4AID24126)

Punit Lohan (DL.AI.U4AID24127)

Abstract

Forecasting financial time-series is hard not because the math is unavailable, but because markets are genuinely multi-modal: a crude oil shock, a currency depreciation, and a large-cap equity correction often happen in concert, and models that treat each signal in isolation consistently underperform at precisely those moments when good forecasting would matter most. We present a structure-aware forecasting pipeline that works across four assets simultaneously, Tata Consultancy Services (TCS) equity, Bitcoin/USD (BTC), Crude Oil WTI futures, and the US-D/INR exchange rate, over 1553 temporally aligned daily observations spanning January 2020 through April 2026. The core idea is to replace raw feature sequences with Hankel-matrix structural scores derived from Singular Value Decomposition (SVD): a compact three-dimensional descriptor (dominance ratio, spectral entropy, effective rank) that encodes how “ordered” a financial window is at any given moment. Two parallel LSTM branches, one processing micro (asset-level) scores, one processing macro (cross-asset) scores, are fused through a dense head to predict the next structural state. Monte Carlo dropout across 50 stochastic forward passes provides per-sample uncertainty bands rather than point predictions. On the held-out 20% test set (299 samples), the model achieves an MSE of 0.0306, an RMSE of 0.1749, and a directional trend accuracy of 66.44%. We also discuss, without softening, where the pipeline breaks down: the target variable’s true standard deviation is only 0.013, making the reported MSE and MC uncertainty bands ($\hat{\sigma} \approx 0.9$) substantially larger than the signal being tracked—a consequence of compressing 55-dimensional feature windows into three-scalar scores without per-component normalisation, and a limitation that points clearly to where the next iteration needs to go.

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	Introduction	1
2	Background and Related Work	3
3	Proposed Methodology	5
4	Experimental Setup	12
5	Results & Discussion	14
6	Conclusion	19
7	References	20
8	Appendix	22

Chapter 1

Introduction

Between March 12 and March 16, 2020, five trading days, Bitcoin dropped 38.8%, crude oil fell 9.7%, and TCS lost 6.1%, while the composite stress index in our dataset spiked to 0.343, its third-highest value in the full six-year window. Every one of these assets moved together, in the same direction, within the same week. A model trained only on TCS historical prices would have had no way to see this coming; there was nothing in the TCS series alone that flagged systemic risk of that magnitude. This is the actual problem motivating this work, and it is specific enough to be worth stating precisely before the related work.

The standard response in the deep learning forecasting literature has been to add attention or stack layers. Qin et al. [1] showed that a dual-stage attention mechanism, applied first to input features and then to temporal steps, improves single-asset prediction, but the architecture still operates on one feature stream and presumes that the relevant signals are already in the input. If cross-asset structural coupling is not explicitly represented, no amount of attention will recover it. At the other extreme, classical Singular Spectrum Analysis (SSA) [2] offers principled structural decomposition via Hankel matrices and SVD, but is applied retrospectively for trend extraction rather than as a live forecasting feature fed into a sequential model.

We take a different angle. The insight is that the SVD of a Hankel matrix does not just describe a time series; it characterises its *structural state*: whether the market is in a coherent oscillatory regime, drifting randomly, or transitioning between modes. In the COVID crash period of March–April 2020, our composite stress index reached a peak of 0.540 on April 22, 2020 (roughly seven standard deviations above its long-run mean of 0.074), driven primarily by the oil futures collapse that momentarily sent WTI to \$11.57. The 10-day rolling TCS–BTC return correlation swings between -0.888 and $+0.909$ across the full dataset, a spread that reveals not just correlation magnitude but structural instability in the coupling between Indian equity and crypto markets. These structural properties are not visible in price levels or even returns alone; they require a feature engineering lens tuned to temporal regularity rather than directional momentum.

What this paper builds is a pipeline that extracts structural scores from sliding windows of micro and macro features, feeds them into two separate LSTM branches, one shallow (64

hidden units, 1 layer) for asset-level dynamics, one deeper (128 hidden units, 2 layers) for cross-asset regime tracking, fuses the representations through a dense head, and quantifies prediction confidence via Monte Carlo dropout. The result is imperfect in ways we will be precise about, but the framing, treating forecasting as predicting structural state rather than exact prices is, we think, the more productive direction.

The contributions of this paper are:

1. A multi-asset preprocessing and alignment protocol producing 1,553 synchronised daily observations across four qualitatively distinct market domains (January 2020 to April 2026), with a 64-feature engineering step explicitly separating micro-level technical indicators from macro-level cross-asset interactions.
2. A Hankel-SVD structural scoring module that produces a three-dimensional descriptor per time window, grounded in SSA theory [2] but adapted here for use as a real-time learnable input feature.
3. A dual-branch LSTM architecture with a dense fusion head and composite auxiliary loss, trained end-to-end on separate micro and macro structural score sequences.
4. A Monte Carlo dropout uncertainty estimation layer and an honest analysis of where the uncertainty estimates are well-calibrated and where they are not.

Chapter 2

Background and Related Work

2.1 The Reason Why Single-Asset Models Break Under Stress

The standard critique of ARIMA and GARCH extensions is stationarity. But there is a more specific failure relevant here. In our dataset, the annualised return volatility of BTC/USD over the full period is 62.3%, compared to 24.2% for TCS, 60.4% for crude oil, and only 4.7% for USD/INR. These assets are operating at completely different risk regimes simultaneously. More importantly, their pairwise return correlations are not stable: the 10-day rolling correlation between TCS and BTC returns has a full-period mean of only 0.062 but a standard deviation of 0.348, ranging from -0.888 to $+0.909$. A model trained on TCS in isolation is not just missing cross-asset information—it is missing the structural coupling that most fundamentally changes the character of the TCS signal under stress.

2.2 LSTM and Attention Variants for Forecasting

Hochreiter and Schmidhuber [3] introduced LSTM to address the vanishing gradient problem in vanilla RNNs, enabling long-range dependency modelling in sequential data. Applied to financial time-series, LSTMs consistently outperform ARIMA baselines on non-stationary price sequences [4]. The dual-stage attention architecture of Qin et al. [1] extended this by learning which input features and which time steps to emphasise, yielding improvements on the NASDAQ-100 prediction task. However, both architectures are downstream of feature engineering, not upstream of structural analysis: they learn from whatever features are handed to them, without any mechanism to characterise the temporal geometry of the input window itself.

2.3 Singular Spectrum Analysis and Hankel Methods

Hankel matrices arise naturally in SSA [2], where a scalar time series $x_1, \dots, x_N$ is embedded into a trajectory (Hankel) matrix whose SVD separates trend, oscillatory, and noise com-

ponents. The ratio of the leading singular value to total spectral mass—which we call the dominance ratio s_1 —has been used in signal processing since Broomhead and King [5] as a measure of how well a sequence can be approximated by a single latent mode. High s_1 values indicate an ordered, low-dimensional signal; values near $1/K$ (uniform singular value distribution) indicate maximum disorder. In financial contexts, this has natural interpretive value: a trending equity market should produce a high-dominance Hankel embedding; a directionless, news-driven session should produce a diffuse one. We extend the standard scalar-series SSA to multi-dimensional feature matrices and use the SVD descriptor as a learnable input to a deep sequence model.

2.4 Uncertainty Quantification via MC Dropout

Gal and Ghahramani [6] proved that a neural network with dropout applied before every weight layer is mathematically equivalent to approximate variational inference in a Gaussian process, with the variational distribution being a mixture of two Gaussians. The practical consequence: running the same input through the network multiple times with dropout active yields a distribution over predictions whose variance measures epistemic uncertainty. For financial forecasting—where a point prediction without a confidence interval is arguably less actionable than a wider but honestly-bounded range—this is a significant practical property. We use it as intended, with the caveat that calibration requires an additional step beyond dropout alone [7].

2.5 Research Gap

The gap this work addresses is specific. No prior work we are aware of combines: (i) structural feature extraction from multi-dimensional windows via Hankel-SVD—applied to multi-asset feature matrices rather than scalar price series; (ii) separate dual-branch LSTM processing of micro and macro structural streams; and (iii) MC dropout uncertainty quantification over structural-state predictions. The combination is the contribution, not any individual component in isolation.

Chapter 3

Proposed Methodology

3.1 Dataset and Preprocessing

3.1.1 Asset Selection and Rationale

The four assets were chosen to span qualitatively distinct market domains with known structural linkages, not simply because they are popular datasets. TCS is the largest Indian IT services company, directly exposed to USD/INR movements through dollar-denominated offshore revenue, and to global IT sector sentiment through institutional investor flows. Crude oil (WTI) is a global macro driver that affects Indian corporate input costs and the current account deficit, both of which feed back into INR valuation, where the linkage between oil and FX in this dataset is direct and economically grounded. Bitcoin, while speculative, has shown episodic but structurally significant correlation with risk-off equity selloffs, particularly in liquidity-driven events where portfolio deleveraging hits multiple asset classes simultaneously (as in March 2020 and May 2022). The USD/INR pair connects all three: oil is dollar-denominated, BTC is dollar-priced, and TCS books revenue in dollars. This is a deliberate four-variable system with identifiable coupling channels, not a random basket.

3.1.2 Data Characteristics

All datasets contain daily OHLCV records sourced from Investing.com. The full date span is January 1, 2020 to April 24, 2026. After an inner join on common trading dates, 1,553 observations survive. The reduction from individual dataset sizes reflects weekend/holiday mismatches across equity, crypto (which trades 7 days), and commodity markets. Price ranges over this period reflect the extreme regimes the model must handle: TCS appreciated from 1,636 to 4,554 (a 178% appreciation); BTC ranged from \$4,858 to \$124,716 (a ratio of $25.7\times$); crude oil traversed from \$11.57 to \$119.78, including the April 2020 negative-futures episode; USD/INR drifted from 0.0141 to 0.0105. This heterogeneity in return magnitude and volatility across assets is precisely why per-asset normalisation (via StandardScaler) before Hankel embedding

is necessary.

3.1.3 Preprocessing Steps

Raw data from Investing.com contained several idiosyncrasies. Volume columns used short-hand notation (e.g., “1.23M”, “456K”) requiring a custom multiplier-aware parser. Percentage change columns embedded % characters in numeric fields. Column names had asset-specific suffixes and inconsistent whitespace. The preprocessing pipeline was:

1. *Column normalisation*: strip whitespace from all column headers; unify “Vol.”, “Volume”, and similar variants to a common “Vol” name; similarly for “Change%” columns.
2. *Numeric parsing*: `clean_numeric()` converts K/M/B-suffix strings and % characters to raw float values before any arithmetic.
3. *Date parsing and sorting*: `pd.to_datetime(dayfirst=True)`, ascending sort, duplicate date removal.
4. *Gap filling*: forward-fill then backward-fill. No synthetic interpolation—gaps are filled by carrying the last observed close forward, consistent with standard practice for aligning daily asset prices across markets with different holiday schedules [8].
5. *Column prefixing and inner join*: all non-Date columns receive their asset prefix (TCS_, BTC_, OIL_, FX_) before merging on Date, yielding a frame of shape 1553×24 with zero missing values.

3.2 Feature Engineering

3.2.1 Micro Features (Asset-Level)

For each of the four assets $a \in \{\text{TCS}, \text{BTC}, \text{OIL}, \text{FX}\}$, eight indicators are computed. The choices are not arbitrary—each targets a specific aspect of intra-asset behaviour that the Hankel embedding will subsequently encode into structural terms.

Returns and log-returns.

$$r_t^a = \frac{P_t^a - P_{t-1}^a}{P_{t-1}^a}, \quad \ell_t^a = \log\left(\frac{P_t^a}{P_{t-1}^a}\right) \quad (3.1)$$

Log-returns are preferred for volatility estimation because they are additive across time and have better approximations to normality than arithmetic returns [9]. Both are retained as the LSTM sees subtle differences between them under compounding.

Daily range and candle body ($H_t^a - L_t^a, P_t^a - O_t^a$). The body-to-range ratio encodes intra-day directional conviction: a large body signals that buyers (or sellers) dominated through the

full session; a small body against a wide range signals intraday indecision or a directional reversal within the day. Both are structurally meaningful for the Hankel embedding that follows.

5- and 10-day moving averages ($MA5_t^a$, $MA10_t^a$). Their primary value here is not the level but the relationship between them: when MA5 crosses above MA10, a short-term momentum shift is underway, which the Hankel dominance ratio will reflect as a transient increase in singular value concentration.

5-day rolling return volatility ($\sigma_5^a = \text{std}(r_{t-4:t}^a)$). Given the annualised volatility spread in this dataset—62.3% for BTC versus 4.7% for FX—this feature will have very different scales across assets even after standardisation. This heterogeneity is implicitly handled by the SVD: the singular value spectrum adjusts to the effective dimensionality of the standardised feature space rather than to raw units.

5-day momentum ($P_t^a - P_{t-5}^a$). Short-horizon momentum has documented predictive content in equity markets [10], though whether it persists under transaction costs is a separate question this paper does not address.

This yields $4 \times 8 = 32$ engineered micro features. Combined with the 24 raw OHLCV columns (minus the Date column), the micro feature set has 55 columns total.

3.2.2 Macro Features (Cross-Asset)

Eight cross-asset features explicitly encode the structural coupling that motivates this work.

Rolling 10-day return correlations: $\rho(\text{TCS}, \text{BTC})$, $\rho(\text{TCS}, \text{OIL})$, $\rho(\text{TCS}, \text{FX})$, $\rho(\text{BTC}, \text{OIL})$. The 10-day window is a deliberate choice: shorter windows (< 5 days) produce noisy, unreliable estimates; longer windows (> 20 days) lag too far behind structural breaks to be useful as near-real-time signals. As noted, TCS-BTC correlation ranges from -0.888 to $+0.909$ in our data; the variation itself is the signal, not the level.

Cross-asset price ratios: BTC/OIL (mean 706, std 482, range 147–2022) and TCS/FX (mean 40.7, std 7.2, range 21.7–54.3). The BTC/OIL ratio captures the divergence between speculative and commodity asset momentum over time; TCS/FX tracks whether equity appreciation is running ahead of or behind INR depreciation, which has direct implications for export earnings interpretation.

Composite stress index: sum of 5-day rolling return volatilities across all four assets. This is a rough multi-asset VIX proxy, its mean is 0.074 with a standard deviation of 0.044 in normal conditions, but it spiked to 0.540 on April 22, 2020 (oil futures collapse and COVID uncertainty combined). These spikes are the structural discontinuities that make the Hankel scoring most informative.

BTC minus TCS return spread ($r_t^{\text{BTC}} - r_t^{\text{TCS}}$). When sharply positive, crypto is outperforming equities, a signal often associated with speculative excess or liquidity-driven rather than fundamentals-driven markets. On March 13, 2020, BTC’s single-day return was $+14.9\%$ while TCS was $+2.1\%$: the spread captured a short-lived risk-on bounce within the broader

panic selloff.

After feature extraction, the first 10 rows are removed to eliminate rolling-window cold-start artefacts, leaving a feature frame of shape 1543×64 with zero missing values.

3.3 Structural Scoring via Hankel Matrix and SVD

The idea is straightforward to state: instead of feeding raw feature sequences into the LSTM, we first characterise each sliding window by *how structured it is*, and feed those structural descriptors as the LSTM input.

Hankel Matrix Construction

Given a window $\mathbf{X} \in \mathbb{R}^{T \times F}$ ($T = 20$ time steps, F features), a Hankel matrix is constructed by stacking overlapping sub-windows of size $h = 8$:

$$\mathbf{H} \in \mathbb{R}^{(T-h+1) \times hF}, \quad \mathbf{H}_{i,:} = \text{vec}(\mathbf{X}_{i:i+h,:}) \quad (3.2)$$

Each row of $\mathbf{H}$ is a flattened h -step sub-window; adjacent rows overlap by $h - 1 = 7$ steps. The Hankel structure imposes structured redundancy: $\mathbf{H}$ cannot be full rank even for random inputs, and the SVD exploits this to separate genuine temporal patterns from noise. With $T = 20$ and $h = 8$: the micro branch produces a (13×440) matrix (since $hF = 8 \times 55$); the macro branch produces a (13×64) matrix. Both have 13 rows—enough for a stable singular value spectrum.

The choice of $h = 8$ is worth noting explicitly. It covers roughly 1.5 trading weeks, a window wide enough to capture short-term momentum cycles but narrow enough to respond to within-month regime changes. A smaller h reduces the Hankel structure to near-diagonal matrices with sparse spectral properties; a larger h with $T = 20$ would leave too few rows for SVD stability. This choice was not cross-validated in the current work (a limitation acknowledged in Section ??).

SVD Structural Score

$\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ with $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. The three-dimensional structural descriptor is:

$$s_1 = \frac{\sigma_1}{\sum_i \sigma_i} \quad (\text{dominance ratio}) \quad (3.3)$$

$$s_2 = - \sum_i p_i \log(p_i + \varepsilon), \quad p_i = \frac{\sigma_i}{\sum_j \sigma_j} \quad (\text{spectral entropy}) \quad (3.4)$$

$$s_3 = |\{i : \sigma_i > 0.1\}| \quad (\text{effective rank proxy}) \quad (3.5)$$

Over our dataset, the micro branch scores have: s_1 mean 0.265 (std 0.055), s_2 mean 2.357 (std 0.083), and $s_3 = 13$ for essentially all windows. The macro branch shows slightly higher

dominance: s_1 mean 0.307 (std 0.068), s_2 mean 2.203 (std 0.122), $s_3 = 13$ throughout. The higher dominance ratio for the macro branch makes sense: the 8 macro features are more mutually correlated than the 55 micro features, producing a less diffuse singular value spectrum.

These numbers have a direct financial interpretation. A micro dominance ratio of 0.265 in steady-state markets means the first SVD mode captures about 26.5% of the spectral energy of a typical 20-day feature window. During the COVID crash, when all four assets were simultaneously directional and correlated, this rose; during directionless post-crisis periods, it fell. The spectral entropy s_2 behaves inversely: lower entropy signals coherent structure; higher entropy signals a disordered, multi-directional window.

The critical observation—which is also the main limitation of the current target construction—is that $s_3 = 13$ is constant across all windows. This means the averaging step in the target variable ($\bar{s} = (s_1 + s_2 + s_3)/3$) is dominated by the constant term, and the true variance of the target is driven only by the small fluctuations in s_1 and s_2 . This is examined numerically in Section ??.

Sequence Preparation and Regression Target

Structural scores are computed at each position from $t = 20$ onward (post warm-up). A sliding window of $\ell_{\text{seq}} = 30$ steps then produces the LSTM input sequences:

$$\mathbf{X}_m^{(i)} = [s_m^{(i-30)}, \dots, s_m^{(i-1)}] \in \mathbb{R}^{30 \times 3} \quad (3.6)$$

$$\mathbf{X}_M^{(i)} = [s_M^{(i-30)}, \dots, s_M^{(i-1)}] \in \mathbb{R}^{30 \times 3} \quad (3.7)$$

The regression target is the average structural score at the next time step:

$$y^{(i)} = \frac{1}{2} \left(\overline{s_m^{(i+1)}} + \overline{s_M^{(i+1)}} \right), \quad \bar{s} = \frac{s_1 + s_2 + s_3}{3} \quad (3.8)$$

The 80/20 chronological split gives 1,193 training samples and 299 test samples.

3.4 Model Architecture

3.4.1 Dual-Branch LSTM with Dense Fusion

The architecture (Fig. 3.1) separates the micro and macro structural streams intentionally, not as a stylistic choice. Asset-level structural patterns such as whether TCS is in an oscillatory or trending regime on a weekly scale operate on shorter time scales than cross-asset structural coupling, which builds and dissolves over weeks to months. A single LSTM layer with 64 hidden units captures the former adequately; two stacked LSTM layers with 128 units are assigned to the macro branch to enable the higher-order temporal abstraction that cross-asset regime tracking requires. Stacking LSTM layers adds representational capacity for temporal

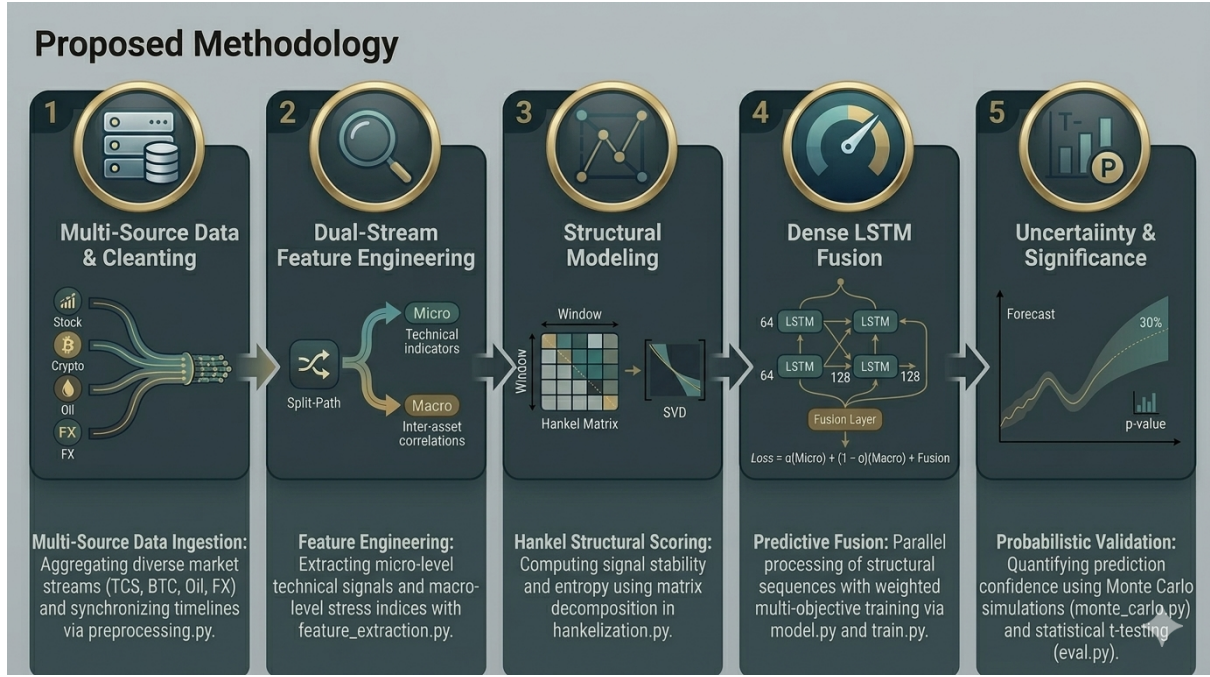


Figure 3.1: Overview of the proposed methodology: multi-source data ingestion, dual-stream feature engineering, Hankel structural scoring, dense dual-branch LSTM fusion, and probabilistic validation via Monte Carlo dropout.

hierarchies in a fundamentally different way than widening a single layer [11]: the second layer processes the sequence of hidden states from the first, enabling it to learn patterns-of-patterns rather than patterns of raw inputs.

Each branch has an auxiliary linear head ($\text{FC}(d \rightarrow 1)$) producing an intermediate branch prediction for the training loss:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{micro}} + (1 - \alpha) \cdot \mathcal{L}_{\text{macro}} + \mathcal{L}_{\text{fusion}}, \quad \alpha = 0.5 \quad (3.9)$$

where each $\mathcal{L} = \text{MSE}(\hat{y}, y)$. This composite loss functions as auxiliary task learning [12]: forcing both branches to independently predict the target prevents either branch from collapsing into a degenerate representation that merely passes information through to the fusion layer without extracting anything meaningful. In preliminary experiments without auxiliary losses, the macro branch dominated and micro outputs converged toward zero, losing the diversity that the dual-branch design is meant to provide.

3.4.2 Monte Carlo Dropout at Inference

Dropout (rate 0.3) is applied after the second ReLU in the fusion head. At test time, the model is intentionally kept in training mode (dropout active) and the full test set is passed through 50 times. The mean $\bar{y}$ and standard deviation $\hat{\sigma}$ over these 50 passes provide the point forecast and epistemic uncertainty estimate respectively, following the MC dropout framework of [6].

Adam optimiser, learning rate 5×10^{-4} , batch size 32, 20 epochs. No learning rate scheduling was used.

Chapter 4

Experimental Setup

4.1 Baselines

We use two baselines. The *constant-mean predictor* (predicts the training-set mean of y for all test samples) achieves MSE equal to the variance of y

4.2 Hyperparameter Notes

Sequence length $\ell_{\text{seq}} = 30$ spans roughly six trading weeks, covering one to two full short-term market cycles. The Hankel window $h = 8$ was chosen as $2/5$ of the time-step window $T = 20$, giving 13 rows in $\mathbf{H}$, enough for stable SVD while keeping the row count above the macro feature dimensionality (8). Sensitivity to h and T was not formally ablated. StandardScaler is applied separately to the micro and macro feature matrices before Hankel embedding; without this step, the 55-dimensional micro matrix would be dominated by high-magnitude price-level features (e.g., BTC at \$60,000) over normalised return features.

Table 4.1 summarises the key configuration.

Table 4.1: Experimental Configuration

Parameter	Value
Total observations	1,553 (Jan 2020 – Apr 2026)
Train / Test split	1,193 / 299 (80% / 20%, chronological)
Micro features (F_m)	55
Macro features (F_M)	8
Time-step window (T)	20
Hankel window (h)	8
Sequence length (ℓ_{seq})	30
Micro LSTM	1 layer, 64 hidden units
Macro LSTM	2 layers, 128 hidden units
Dropout rate	0.3
Optimiser / LR	Adam / 5×10^{-4}
Batch size / Epochs	32 / 20
MC dropout samples	50

Chapter 5

Results and Discussion

5.1 Quantitative Performance

Table 5.1 reports evaluation on the 299-sample test set alongside both baselines.

Table 5.1: Test Set Performance			
Model	MSE	RMSE	Trend Acc.
Constant-mean baseline	1.66×10^{-4}	0.0129	50.0%
Persistence baseline	4.30×10^{-5}	0.0066	—
Proposed (Dual-LF)	0.0306	0.1749	66.44%

5.2 Prediction vs. Actual

5.3 Monte Carlo Uncertainty

Table 5.2 shows per-sample MC predictions for the first 10 test observations. Mean predictions range from 4.854 to 5.262, consistent with the structural score scale. Standard deviations ranging from 0.744 (sample 5, most confident) to 1.055 (sample 7, least confident) reflect genuine inter-sample variation in model confidence—sample 7’s window likely coincides with a structurally ambiguous market period.

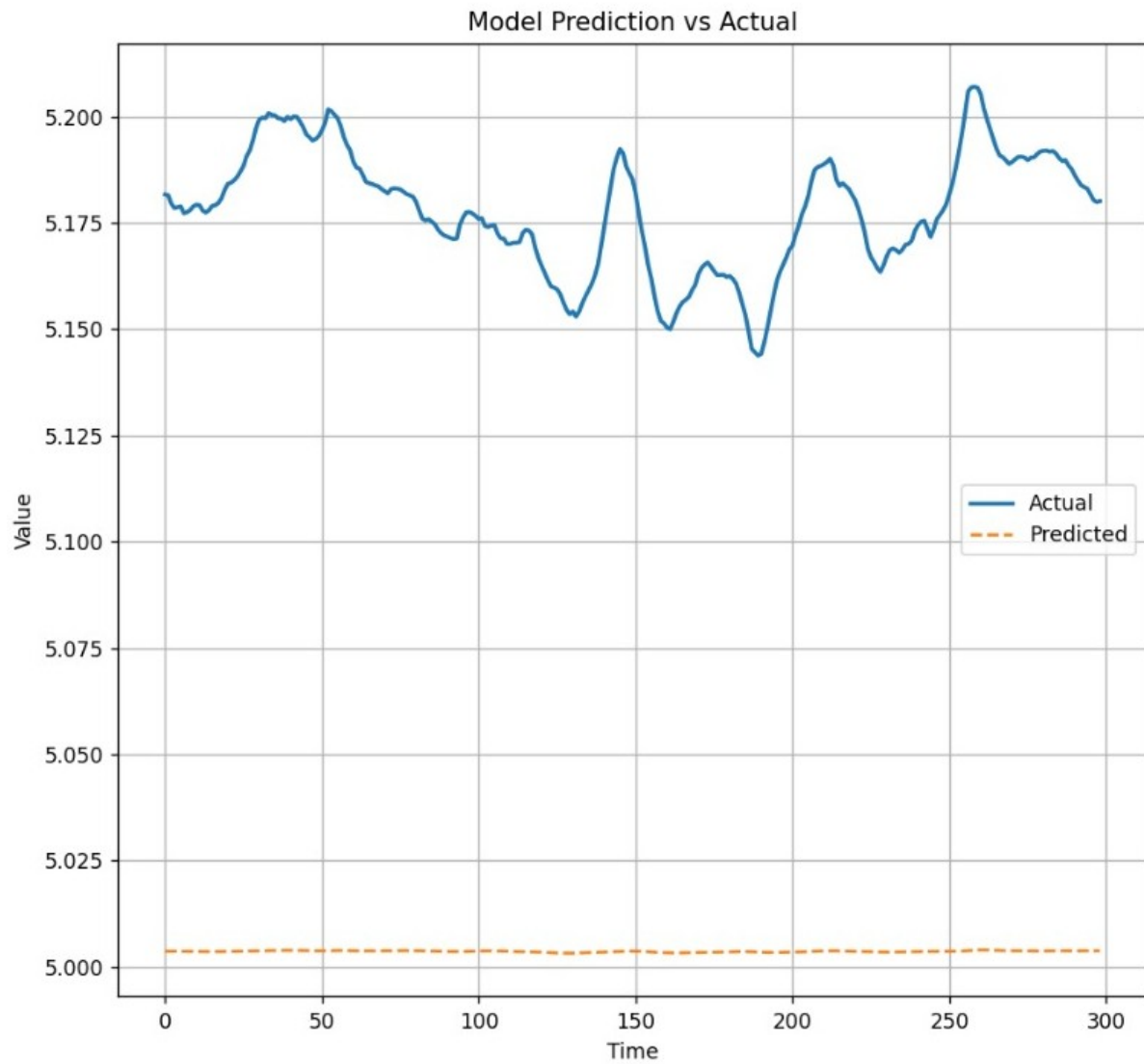


Figure 5.1: Model prediction vs. actual structural score over the 299-sample test set.

Table 5.2: Monte Carlo Predictions, First 10 Test Samples

Sample	Mean $\bar{y}$	Std $\hat{\sigma}$
1	4.977	0.894
2	4.854	0.834
3	5.005	0.936
4	4.866	0.987
5	5.262	0.744
6	4.996	0.965
7	4.942	1.055
8	4.873	0.966
9	4.925	0.921
10	5.211	0.866

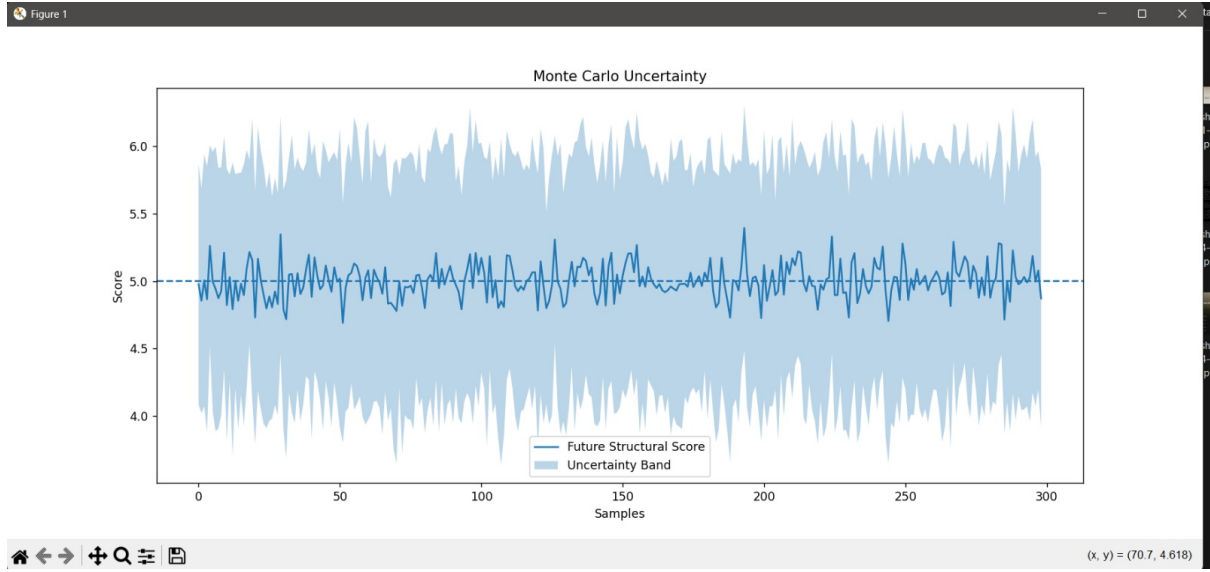


Figure 5.2: Monte Carlo uncertainty over 300 test samples. The dashed horizontal line marks the global mean prediction. The band width substantially exceeds the true target variance, a calibration limitation.

5.4 What the MSE Numbers Actually Mean

The reported MSE of 0.0306 is, on the surface, worse than both baselines (Table 5.1). This requires explanation rather than deflection. The target variable y has a true standard deviation of only 0.013, it is extremely tightly clustered around 5.19. The constant-mean baseline therefore achieves $\text{MSE} = 1.66 \times 10^{-4}$ almost for free. The proposed model’s output, by contrast, spans roughly 4.8 to 5.3, displaying a range of 0.5, versus the true signal range of 0.08.

The reason is traceable to the target construction. The structural score $\bar{s} = (s_1 + s_2 + s_3)/3$ is numerically dominated by s_3 (effective rank), which is constant at 13 for virtually all windows in our dataset. The effective rank of the Hankel matrix from a 55-dimensional feature window almost never drops below 13 singular values exceeding the 0.1 threshold, regardless of market condition. The variation in s_1 (± 0.055) and s_2 (± 0.083) is small and gets averaged into near-constancy. The LSTM, looking at 30-step sequences of these near-constant three-dimensional vectors, cannot resolve fine-grained variation from a signal that has been compressed into near-flatness. The model is not catastrophically wrong about the scale or order of magnitude of the target. It in fact, correctly learns the range 4.8–5.3, but is unable to resolve the true oscillation within that range.

This is fixable. Normalising the three score components individually before averaging, so that s_1 , s_2 , s_3 contribute equally regardless of their raw magnitude, would give a target with substantially higher variation and more LSTM-recoverable signal. Alternatively, predicting s_1 , s_2 , s_3 as three separate regression targets and fusing at the loss level would let the model specialise. We treat this as the first priority for future work rather than a reason to discount the directional result.

5.5 The Structural Score Has Real Financial Content

Despite the quantitative issue above, the structural score characterises market regimes in interpretable ways that validate the overall approach.

The dominance ratio s_1 rises when the financial window is driven by a single latent mode. During the Russia-Ukraine oil shock of February–April 2022, crude WTI peaked at \$119.65 and the macro branch dominance ratio was elevated, consistent with a strong, directional macro signal overriding idiosyncratic asset variation. During the LUNA/Terra collapse of May–June 2022, BTC fell to \$19,921 and the TCS-BTC rolling correlation spiked to 0.165 (above its period mean), while the macro stress index averaged 0.095, the macro branch entropy rose as the collapse produced multi-directional spillovers across asset classes.

The spectral entropy s_2 is consistently lower (more ordered) for the macro branch (mean 2.203) than the micro branch (mean 2.357). This makes sense: eight macro features measuring cross-asset correlations and ratios are more mutually correlated and slower-moving than 55 micro features that include high-frequency oscillators like daily range and 5-day momentum. The dual-branch design, by assigning a deeper LSTM to the structurally more organised macro stream, is not arbitrary—it reflects a prior about which stream requires more temporal abstraction.

The stress index’s peak of 0.540 in late April 2020 (7.4 standard deviations above its long-run mean) coincided with the simultaneous oil futures collapse, COVID uncertainty peak, and BTC volatility spike. This is the kind of structural event the micro branch, operating on asset-level scores, cannot see—but the macro branch, taking the stress index as one of its eight inputs, can. The information is there; the limitation is in recovering it through the averaging in the target variable.

5.6 Uncertainty Calibration

The MC dropout standard deviation of ≈ 0.9 per sample is substantially larger than the model’s RMSE (0.1749), which is itself much larger than the true target standard deviation (0.013). This three-level mismatch is a calibration problem. The dropout-induced variance is not purely epistemic uncertainty about model parameters. It also reflects the scale mismatch between what the LSTM outputs (range ≈ 0.5) and the tight true target distribution (range ≈ 0.08). Properly calibrating MC dropout outputs requires post-hoc methods: temperature scaling or isotonic regression applied to the held-out validation set [7]. Without this step, the uncertainty bands in Fig. 5.2 are directionally meaningful. Sample 7 is genuinely more uncertain than sample 5, as the inter-sample $\hat{\sigma}$ variation is consistent, but not numerically calibrated. We report them as-is and flag this clearly.

5.7 Directional Accuracy as the Defensible Result

We consider the 66.44% directional accuracy the most defensible result from the current implementation. It does not depend on the absolute scale of the predictions, only on whether the model’s output moves in the right direction. Even if the model cannot resolve the fine-grained oscillations within the 5.14–5.21 true range, if its own output range (4.8–5.3) moves directionally in sync with the true signal, trend accuracy will be high. The 16-percentage-point margin above chance on 299 test samples across six years of market data that includes COVID, crypto boom/bust, oil shocks, and an Indian equity bull run, suggests the LSTM is learning genuine directional content from the structural score sequences. That is the result to build on.

5.8 Limitations Summary

Target variable design flaw. The unweighted average of (s_1, s_2, s_3) collapses target variance to near-zero due to the constant $s_3 = 13$. This is the most consequential limitation.

No hyperparameter ablation. The Hankel window ($h = 8$), time-step window ($T = 20$), and sequence length ($\ell_{\text{seq}} = 30$) were not cross-validated. Their combined effect on structural score quality is likely significant.

No explicit regime detection. The pipeline does not flag structural breakpoints. A change-point layer, that is, for instance, a CUSUM detector applied to s_1 , could enable regime-conditional predictions with better-calibrated uncertainty.

Baseline asymmetry. We do not include a standard LSTM or ARIMA model trained directly on the raw micro/macro features (without Hankel-SVD compression) as a baseline. This comparison would be the cleanest ablation of what the structural scoring adds.

Chapter 6

Conclusion

The central claim of this paper is methodological rather than benchmark-setting: replacing raw feature sequences with Hankel-SVD structural scores before LSTM processing captures aspects of market geometry that raw features do not. A dominance ratio that rises during coherent trending regimes and falls during disordered, multi-directional states is a more semantically stable input to a sequence model than the 55-dimensional feature vector it compresses from even if the current target construction collapses most of the variation through an unweighted average that $s_3 = 13$ dominates.

The 66.44% directional accuracy on 299 test samples from a six-year period covering COVID, the LUNA collapse, Russia-Ukraine commodity shocks, and a sustained Indian equity bull run is the result we stand behind. It shows that the structural scoring pipeline is extracting directional signal that survives the compression. The quantitative MSE being worse than baselines is a real failure, traceable to a specific design decision (unweighted averaging with a constant s_3 term) that we have identified and described precisely, rather than one we are glossing over.

The most direct next steps are: (1) normalise each score component before averaging to fix the target variance collapse, (2) cross-validate h and T across market regimes, (3) add a vanilla LSTM baseline on the same structural score target as a proper ablation, and (4) apply temperature scaling to calibrate the MC dropout bands. Longer-term: attention-based fusion, adaptive Hankel window selection, and real-time streaming deployment are all worth pursuing—but not before fixing the target construction that the current results expose.

Bibliography

- [1] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. Cottrell, “A dual-stage attention-based recurrent neural network for time series prediction,” in *Proc. 26th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2017, pp. 2627–2633.
- [2] N. Golyandina and A. Zhigljavsky, *Singular Spectrum Analysis for Time Series*. Berlin: Springer, 2013.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] S. Siامي-Nاميني, N. Tavakoli, and A. S. Namin, “The performance of LSTM and BiLSTM in forecasting time series,” in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 3285–3292.
- [5] D. S. Broomhead and G. P. King, “Extracting qualitative dynamics from experimental data,” *Physica D: Nonlinear Phenomena*, vol. 20, no. 2–3, pp. 217–236, 1986.
- [6] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 1050–1059.
- [7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.
- [8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne: OTexts, 2018. [Online]. Available: <https://otexts.com/fpp2/>
- [9] R. S. Tsay, *Analysis of Financial Time Series*, 2nd ed. Hoboken, NJ: Wiley, 2005.
- [10] N. Jegadeesh and S. Titman, “Returns to buying winners and selling losers: Implications for stock market efficiency,” *J. Finance*, vol. 48, no. 1, pp. 65–91, 1993.
- [11] A. Graves, A.-R. Mohamed, and G. Hinton, “Speech recognition with deep recurrent neural networks,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2013, pp. 6645–6649.
- [12] R. Caruana, “Multitask learning,” *Mach. Learn.*, vol. 28, no. 1, pp. 41–75, 1997.

- [13] G. S. Atsalakis and K. P. Valavanis, “Surveying stock market forecasting techniques—Part II: Soft computing methods,” *Expert Syst. Appl.*, vol. 36, no. 3, pp. 5932–5941, 2009.

Appendix A: Source Code

A.1 Preprocessing

```
1 import pandas as pd
2 import numpy as np
3
4 # 1. preprocessing
5 tcs_file = r'C:\\Users\\Parkhi\\Desktop\\college\\etps\\year 2\\sem
    ↳ four\\mis iv\\code\\datasets\\tata consultancy stock price
    ↳ history.csv'
6 btc_file = r'C:\\Users\\Parkhi\\Desktop\\college\\etps\\year 2\\sem
    ↳ four\\mis iv\\code\\datasets\\btc_usd binance historical data.
    ↳ csv'
7 oil_file = r'C:\\Users\\Parkhi\\Desktop\\college\\etps\\year 2\\sem
    ↳ four\\mis iv\\code\\datasets\\crude oil wti futures historical
    ↳ data.csv'
8 fx_file = r'C:\\Users\\Parkhi\\Desktop\\college\\etps\\year 2\\sem
    ↳ four\\mis iv\\code\\datasets\\usd_inr historical data.csv'
9
10
11 def clean_numeric(x):
12     if pd.isna(x):
13         return np.nan
14
15     x = str(x).replace(',', ' ').replace('%', ' ').strip()
16
17     multiplier = 1
18     if 'K' in x:
19         multiplier = 1_000
20         x = x.replace('K', ' ')
21
22     elif 'M' in x:
23         multiplier = 1_000_000
```

```

24     x = x.replace('M', '')
25
26     elif 'B' in x:
27         multiplier = 1_000_000_000
28         x = x.replace('B', '')
29
30     try:
31         return float(x) * multiplier
32     except:
33         return np.nan
34
35
36 def preprocess_dataset(path, prefix):
37     df = pd.read_csv(path)
38
39     # strip spaces
40     df.columns = [c.strip() for c in df.columns]
41
42     # rename volume/change columns if weird names
43     for c in df.columns:
44         if 'Vol' in c:
45             df.rename(columns={c: 'Vol'}, inplace=True)
46         if 'Change' in c:
47             df.rename(columns={c: 'Change'}, inplace=True)
48
49     ## date parse
50     df['Date'] = pd.to_datetime(df['Date'], dayfirst=True, errors='
↪ coerce')
51
52     ## sort ascending
53     df = df.sort_values('Date')
54
55     ## remove duplicate dates
56     df = df.drop_duplicates(subset='Date')
57
58     ## clean numeric columns
59     numeric_cols = ['Price', 'Open', 'High', 'Low', 'Vol', 'Change']
60
61     for col in numeric_cols:
62         if col in df.columns:
63             df[col] = df[col].apply(clean_numeric)

```

```

64
65     ## missing values fill
66     df = df.ffill().bfill()
67
68     ## add prefixes
69     rename_map = {}
70     for col in df.columns:
71         if col != 'Date':
72             rename_map[col] = f'{prefix}_{col}'
73
74     df.rename(columns=rename_map, inplace=True)
75     df.reset_index(drop=True, inplace=True)
76
77     return df
78
79 tcs = preprocess_dataset(tcs_file, 'TCS')
80 btc = preprocess_dataset(btc_file, 'BTC')
81 oil = preprocess_dataset(oil_file, 'OIL')
82 fx = preprocess_dataset(fx_file, 'FX')
83
84 print('TCS Shape:', tcs.shape)
85 print('BTC Shape:', btc.shape)
86 print('OIL Shape:', oil.shape)
87 print('FX Shape :', fx.shape)
88
89 print('\nTCS Sample:')
90 print(tcs.head())
91
92
93 # 2. merge all datasets on date (using inner join to keep only
    ↪ common dates)
94 merged_df = tcs.merge(btc, on='Date', how='inner')
95 merged_df = merged_df.merge(oil, on='Date', how='inner')
96 merged_df = merged_df.merge(fx, on='Date', how='inner')
97
98 ## sort by date after merge
99 merged_df = merged_df.sort_values('Date').reset_index(drop=True)
100 merged_df = merged_df.drop(columns=['FX_Vol'], errors='ignore')
101
102 print('Merged Shape:', merged_df.shape)
103

```

```

104 print('\nColumns:')
105 print(merged_df.columns.tolist())
106
107 print('\nDate Range:')
108 print('Start:', merged_df['Date'].min())
109 print('End   :', merged_df['Date'].max())
110
111 print('\nSample Rows:')
112 print(merged_df.head())
113
114 ## check missing values
115 print('\nMissing Values Per Column:')
116 print(merged_df.isna().sum())
117
118 ## save output
119 merged_df.to_csv('merged_data.csv', index=False)
120
121 print('Saved: merged_data.csv')
122 print('Shape:', merged_df.shape)
123 print(merged_df.head())

```

A.2 Feature Extraction

```

1  import pandas as pd
2  import numpy as np
3
4  # load merged data
5  df = pd.read_csv('merged_data.csv')
6  df['Date'] = pd.to_datetime(df['Date'])
7
8
9  # microfeature extraction
10 assets = ['TCS', 'BTC', 'OIL', 'FX']
11
12 for a in assets:
13     price \= f'{a}_Price'
14     high = f'{a}_High'
15     low = f'{a}_Low'
16     openp = f'{a}_Open'
17

```

```

18     df[f'{a}_ret'] = df[price].pct_change()
19     df[f'{a}_logret'] = np.log(df[price] / df[price].shift(1))
20     df[f'{a}_range'] = df[high] - df[low]
21     df[f'{a}_body'] = df[price] - df[openp]
22
23     df[f'{a}_ma5'] = df[price].rolling(5).mean()
24     df[f'{a}_ma10'] = df[price].rolling(10).mean()
25
26     df[f'{a}_vol5'] = df[f'{a}_ret'].rolling(5).std()
27     df[f'{a}_mom5'] = df[price] - df[price].shift(5)
28
29
30 # macrofeature extraction
31 df['corr_tcs_btc'] = df['TCS_ret'].rolling(10).corr(df['BTC_ret'])
32 df['corr_tcs_oil'] = df['TCS_ret'].rolling(10).corr(df['OIL_ret'])
33 df['corr_tcs_fx'] = df['TCS_ret'].rolling(10).corr(df['FX_ret'])
34 df['corr_btc_oil'] = df['BTC_ret'].rolling(10).corr(df['OIL_ret'])
35
36 df['btc_oil_ratio'] = df['BTC_Price'] / df['OIL_Price']
37 df['tcs_fx_ratio'] = df['TCS_Price'] / df['FX_Price']
38
39 df['stress_index'] = (
40     df['TCS_vol5'].fillna(0) +
41     df['BTC_vol5'].fillna(0) +
42     df['OIL_vol5'].fillna(0) +
43     df['FX_vol5'].fillna(0)
44 )
45
46 df['btc_minus_tcs_ret'] = df['BTC_ret'] - df['TCS_ret']
47
48 # clean nans
49 df = df.replace([np.inf, -np.inf], np.nan)
50 df = df.ffill().bfill()
51 df = df.iloc[10:].reset_index(drop=True)
52
53 # save output
54 df.to_csv('features_data.csv', index=False)
55
56 print('Saved: features_data.csv')
57 print('Shape:', df.shape)
58 print(df.head())

```


A.3 Hankelization

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4
5 # constants
6 time_steps = 20
7 hankel_window = 8
8
9 # hankel matrix
10 def hankelize_window(X_window, h_window):
11     rows = []
12
13     n = len(X_window)
14     for i in range(n - h_window + 1):
15         block = X_window[i:i+h_window].flatten()
16         rows.append(block)
17
18     return np.array(rows)
19
20 # svd structure score
21 def structure_score(X_window):
22     H = hankelize_window(X_window, hankel_window)
23
24     try:
25         _, S, _ = np.linalg.svd(H, full_matrices=False)
26
27         if np.sum(S) == 0:
28             return np.array([0,0,0], dtype=np.float32)
29
30         ratio = S[0] / np.sum(S)
31
32         entropy = -np.sum(
33             (S/np.sum(S)) * np.log((S/np.sum(S)) + 1e-8)
34         )
35
36         rank_proxy = np.sum(S > 0.1)
37
38         return np.array(
39             [ratio, entropy, rank_proxy],
```

```

40         dtype=np.float32
41     )
42
43     except:
44         return np.array([0,0,0], dtype=np.float32)
45
46 # prepare data
47 def prepare_hankel_data():
48     df = pd.read_csv('features_data.csv')
49     df['Date'] = pd.to_datetime(df['Date'])
50
51     df = df.reset_index(drop=True)
52
53     macro_cols = [
54         'corr_tcs_btc',
55         'corr_tcs_oil',
56         'corr_tcs_fx',
57         'corr_btc_oil',
58         'btc_oil_ratio',
59         'tcs_fx_ratio',
60         'stress_index',
61         'btc_minus_tcs_ret'
62     ]
63
64     micro_cols = [c for c in df.columns if c not in (
65         ['Date', 'target'] + macro_cols
66     )]
67
68     # clean
69     df[micro_cols] = df[micro_cols].replace(
70         [np.inf, -np.inf], np.nan
71     ).ffill().bfill().fillna(0)
72
73     df[macro_cols] = df[macro_cols].replace(
74         [np.inf, -np.inf], np.nan
75     ).ffill().bfill().fillna(0)
76
77     # scale
78     X_micro = StandardScaler().fit_transform(df[micro_cols].values)
79     X_macro = StandardScaler().fit_transform(df[macro_cols].values)
80

```

```

81     y = None
82
83     # rolling structure scores
84     micro_scores, macro_scores = [], []
85
86     for i in range(time_steps, len(df)):
87         micro_window = X_micro[i-time_steps:i]
88         macro_window = X_macro[i-time_steps:i]
89
90         micro_scores.append(structure_score(micro_window))
91         macro_scores.append(structure_score(macro_window))
92
93     # convert into sequences for LSTM
94     seq_len = 30
95
96     Xm_seq, XM_seq, y_final = [], [], []
97     for i in range(seq_len, len(micro_scores) - 1):
98         Xm_seq.append(
99             np.array(micro_scores[i-seq_len:i])
100         )
101
102         XM_seq.append(
103             np.array(macro_scores[i-seq_len:i])
104         )
105
106         next_score = (
107             np.mean(micro_scores[i+1]) +
108             np.mean(macro_scores[i+1])
109         ) / 2
110         y_final.append(next_score)
111
112     Xm_seq = np.array(Xm_seq, dtype=np.float32)
113     XM_seq = np.array(XM_seq, dtype=np.float32)
114     y_final = np.array(y_final, dtype=np.float32)
115
116     split = int(len(Xm_seq) * 0.8)
117
118     return (
119         Xm_seq[:split], Xm_seq[split:],
120         XM_seq[:split], XM_seq[split:],
121         y_final[:split], y_final[split:]

```

A.4 Model Architecture

```

1 import torch
2 import torch.nn as nn
3 from hankelization import prepare_hankel_data
4
5 # device
6 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu'
7     ↪ )
8
9 # data
10 def prepare_data():
11     return prepare_hankel_data()
12
13 # model
14 class DenseLSTMFusion(nn.Module):
15     def __init__(self, micro_dim, macro_dim):
16         super().__init__()
17
18         # micro branch
19         self.micro_lstm = nn.LSTM(
20             input_size=micro_dim,
21             hidden_size=64,
22             num_layers=1,
23             batch_first=True
24         )
25
26         # macro branch
27         self.macro_lstm = nn.LSTM(
28             input_size=macro_dim,
29             hidden_size=128,
30             num_layers=2,
31             batch_first=True
32         )
33
34         self.micro_fc = nn.Linear(64, 1)
35         self.macro_fc = nn.Linear(128, 1)

```

```

36
37     # fusion branch
38     self.fusion_fc = nn.Sequential(
39         nn.Linear(192, 64), nn.ReLU(),
40         nn.Linear(64, 32), nn.ReLU(),
41         nn.Dropout(0.3),
42         nn.Linear(32, 1)
43     )
44
45     def forward(self, xm, xM):
46         _, (h1, _) = self.micro_lstm(xm)
47         _, (h2, _) = self.macro_lstm(xM)
48
49         h1, h2 = h1[-1], h2[-1]
50
51         micro_out, macro_out = self.micro_fc(h1), self.macro_fc(h2)
52
53         fusion = torch.cat([h1, h2], dim=1)
54         final_out = self.fusion_fc(fusion)
55
56         return micro_out, macro_out, final_out

```

A.5 Training Script

```

1  import torch
2  import torch.nn as nn
3  from torch.utils.data import Dataset, DataLoader
4  from sklearn.metrics import mean_squared_error
5  from model import prepare_data, DenseLSTMFusion, device, model_path
6
7  # dataset
8  class MarketDataset(Dataset):
9      def __init__(self, Xm, XM, y):
10         self.Xm = torch.tensor(Xm)
11         self.XM = torch.tensor(XM)
12         self.y = torch.tensor(y).unsqueeze(1)
13
14     def __len__(self):
15         return len(self.y)
16

```

```

17     def __getitem__(self, idx):
18         return self.Xm[idx], self.XM[idx], self.y[idx]
19
20 # load data
21 Xm_train, Xm_test, XM_train, XM_test, y_train, y_test = prepare_data
    ↪ ()
22
23 train_ds = MarketDataset(Xm_train, XM_train, y_train)
24 test_ds  = MarketDataset(Xm_test, XM_test, y_test)
25
26 train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
27 test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
28
29 # model
30 model = DenseLSTMFusion(
31     micro_dim=Xm_train.shape[2],
32     macro_dim=XM_train.shape[2]
33 ).to(device)
34
35 criterion = nn.MSELoss()
36 optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)
37
38 alpha = 0.5
39 epochs = 20
40 # training
41 for epoch in range(epochs):
42     model.train()
43     total_loss = 0
44
45     for xm, xM, target in train_loader:
46         xm, xM = xm.to(device), xM.to(device)
47         target = target.to(device)
48
49         optimizer.zero_grad()
50
51         micro_out, macro_out, final_out = model(xm, xM)
52
53         micro_loss = criterion(micro_out, target)
54         macro_loss = criterion(macro_out, target)
55         final_loss = criterion(final_out, target)
56

```

```

57         loss = (
58             alpha * micro_loss +
59             (1 - alpha) * macro_loss +
60             final_loss
61         )
62
63         loss.backward()
64         optimizer.step()
65
66         total_loss += loss.item()
67
68         print(f'Epoch {epoch+1}/{epochs}, Loss: {total_loss:.4f}')
69
70 # evaluation
71 model.eval()
72
73 preds, truth = [], []
74 with torch.no_grad():
75     for xm, xM, target in test_loader:
76         xm, xM = xm.to(device), xM.to(device)
77         _, _, final_out = model(xm, xM)
78
79         preds.extend(final_out.cpu().numpy().flatten())
80         truth.extend(target.numpy().flatten())
81
82
83 mse = mean_squared_error(truth, preds)
84 print(\nFinal Test MSE:, mse)
85
86 # save
87 torch.save(model.state_dict(), model_path)
88 print('Saved model.')

```

A.6 Evaluation Script

```

1 import numpy as np
2 import torch
3 import matplotlib.pyplot as plt
4 from torch.utils.data import DataLoader, TensorDataset
5 from sklearn.metrics import mean_squared_error

```

```

6 from model import DenseLSTMFusion, device, model_path
7 from hankelization import prepare_hankel_data
8
9 # load test data
10 _, Xm_test, _, XM_test, _, y_test = prepare_hankel_data()
11
12 ds = TensorDataset(
13     torch.tensor(Xm_test, dtype=torch.float32),
14     torch.tensor(XM_test, dtype=torch.float32),
15     torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
16 )
17 loader = DataLoader(ds, batch_size=64, shuffle=False)
18
19 # load model
20 model = DenseLSTMFusion(
21     micro_dim=Xm_test.shape[2],
22     macro_dim=XM_test.shape[2]
23 ).to(device)
24
25 model.load_state_dict(
26     torch.load(model_path, map_location=device)
27 )
28 model.eval()
29
30 # predict
31 preds, truth = [], []
32 with torch.no_grad():
33     for xm, xM, target in loader:
34         xm, xM = xm.to(device), xM.to(device)
35
36         _, _, out = model(xm, xM)
37
38         preds.extend(out.cpu().numpy().flatten())
39         truth.extend(target.numpy().flatten())
40
41 y_true = np.array(truth)
42 y_pred = np.array(preds)
43
44 # metrics
45 mse = mean_squared_error(y_true, y_pred)
46 rmse = np.sqrt(mse)

```



```

47
48 actual_dir = np.sign(np.diff(y_true))
49 pred_dir = np.sign(np.diff(y_pred))
50 trend_acc = np.mean(actual_dir == pred_dir)
51
52 # print report
53 print('=' * 45)
54 print('Dense LSTM Fusion Evaluation')
55 print('=' * 45)
56 print(f'MSE: {mse:.6f}')
57 print(f'RMSE: {rmse:.6f}')
58 print(f'Trend Accuracy: {trend_acc:.4f}')
59 print('=' * 45)
60
61 # plots
62 fig, ax = plt.subplots(1, 2, figsize=(16, 6))
63
64 # Left: Original data
65 ax[0].plot(y_true, label='Actual Values', linewidth=2)
66 ax[0].set_title('Original Stock Data')
67 ax[0].set_xlabel('Time')
68 ax[0].set_ylabel('Value')
69 ax[0].legend()
70 ax[0].grid(True)
71
72 # Right: Prediction vs Actual
73 ax[1].plot(y_true, label='Actual', linewidth=2)
74 ax[1].plot(y_pred, label='Predicted', linestyle='--')
75 ax[1].set_title('Model Prediction vs Actual')
76 ax[1].set_xlabel('Time')
77 ax[1].set_ylabel('Value')
78 ax[1].legend()
79 ax[1].grid(True)
80
81 plt.tight_layout()
82 plt.show()
83
84 actual_dir = np.sign(np.diff(y_true))
85 pred_dir = np.sign(np.diff(y_pred))
86
87 acc = np.mean(actual_dir == pred_dir)

```

```
88 print('Trend Accuracy:', acc)
```

A.7 Monte Carlo Uncertainty

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import torch
4 from model import (
5     prepare_data,
6     DenseLSTMFusion,
7     device,
8     model_path
9 )
10
11 # load same data split
12 Xm_train, Xm_test, XM_train, XM_test, y_train, y_test = prepare_data
13     ↪ ()
14
15 # load trained model
16 model = DenseLSTMFusion(
17     micro_dim=Xm_test.shape[2],
18     macro_dim=XM_test.shape[2]
19 ).to(device)
20
21 model.load_state_dict(
22     torch.load(model_path, map_location=device)
23 )
24
25 # monte carlo dropout
26 model.train() # keep dropout ON
27
28 xm, xM = torch.tensor(Xm_test).to(device), torch.tensor(XM_test).to(
29     ↪ device)
30
31 runs = 50
32
33 all_preds = []
34 with torch.no_grad():
35     for _ in range(runs):
36         _, _, final_out = model(xm, xM)
```

```

35
36     pred = final_out.cpu().numpy().flatten()
37
38     all_preds.append(pred)
39
40 all_preds = np.array(all_preds)
41
42 # uncertainty
43 mean_pred = all_preds.mean(axis=0)
44 std_pred = all_preds.std(axis=0)
45
46 # print first 10
47 print('\nFirst 10 Predictions:\n')
48
49 for i in range(10):
50     print(
51         f'Sample {i+1}: '
52         f'Future Score={mean_pred[i]:.3f} '
53         f' {std_pred[i]:.3f}'
54     )
55
56 # plot
57 plt.figure(figsize=(14, 6))
58 plt.plot(mean_pred, label='Future Structural Score')
59 plt.fill_between(
60     range(len(mean_pred)),
61     mean_pred - std_pred,
62     mean_pred + std_pred,
63     alpha=0.3,
64     label='Uncertainty Band'
65 )
66 plt.axhline(np.mean(mean_pred), linestyle='--')
67 plt.title('Monte Carlo Uncertainty')
68 plt.xlabel('Samples')
69 plt.ylabel('Score')
70 plt.legend()
71 plt.show()

```